Benchmarking MicroBlaze-V CPU Implementations using a Sudoku Solver Algorithm

Christopher Co Jr., Allen Nguyen, Ryan Lui, Nathan Bernal, *Student, California Polytechnic University - Pomona*

***Abstract*— This report examines various implementations of a softcore MicroBlaze-V central processor unit (CPU) on a Nexys A7 Field Programmable Gate Array (FPGA). Comparisons between implementations are performed using a custom-made benchmark utilizing a back-tracking brute force Sudoku algorithm. Key metrics such as the number of clock cycles, the overall program execution time, and throughput are the primary measures of quantitative performance. The number of physical resources the board needs such as Look Up Tables (LUT) and Flip Flops (FF) are secondary metrics used to examine tradeoffs between each unique implementation of the MicroBlaze-V CPU. Our findings found that using our Sudoku benchmark, the Throughput optimization as provided in the configuration wizard had the best overall performance compared to other options, while the Frequency option yielded the worst performance.**

***Index Terms*— Benchmarking, FPGA, MicroBlaze-V, Nexys A7, Pipelining, Reduced Instruction Set Computing (RISC-V), Sudoku solver**

I. INTRODUCTION

Field Programmable Gate Arrays (FPGAs) are very powerful in implementing different hardware configurations. By leveraging the flexibility of an FPGA, designers can benchmark different processor designs and configurations before implementing it in silicon. This process informs designers about how different configurations can influence the performance of a given processor, using metrics like computation time, throughput, and hardware complexity. In this paper, we seek to benchmark different configurations of MicroBlaze-V, a soft processor core designed to be implemented in Xilinx FPGAs as shown in Fig. 1.. We seek to compare and verify the different configurations using a novel processing task, which is to solve a series of Sudoku puzzles. We will implement a brute-force algorithm to test the performance and instruction handling speed of the different processor configurations. By using the FPGA, we can quickly and cheaply implement these different processor cores, as well as implement supporting hardware to measure clock cycles and other pertinent benchmarking parameters.

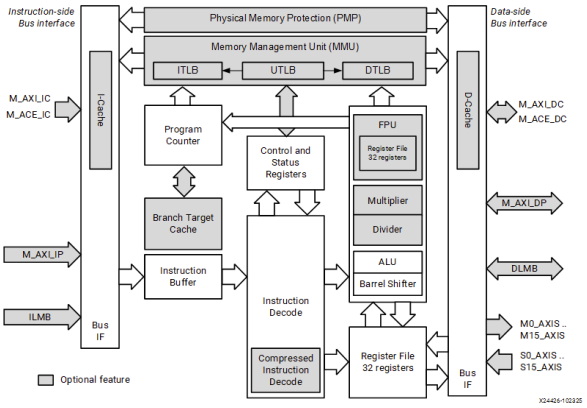

Fig. 1. MicroBlaze block diagram architecture

II. SUDOKU ALGORITHM IMPLEMENTATION

Solving a sudoku board as a human employs many tricks and shortcuts to reduce the amount of time spent. This includes various methods of scanning and analysis, such as the identification of single candidates or naked/hidden pairs. For benchmarking purposes, however, we wish the algorithm to be purposely inefficient to perform as many operations as possible before a potential board could be solved. Thus, a brute force method of solution finding was used to meet these constraints, and this aids in reducing the development complexity of the overall algorithm. Only minor optimizations of the algorithm were introduced including a vector mapping only empty cells to reduce the number of comparisons to cells already filled in at the beginning.

The primary algorithm can be described as follows:

1) Parse a partially filled in 9x9 Sudoku board
2) For each empty cell present in the board, fill it with a digit from 1 to 9 starting with 1
3) Check if the digit is valid by checking the row, column, and 3 x 3 subgrid that cell is a part of.
4) If the digit is valid, continue to the next empty cell.
5) If the digit is invalid, increment to the next possible digit with a max digit of 9.
6) If a cell has iterated through all possible digits of an empty cell and none are valid, return to the last filled empty cell and increment the digit by one.
7) Continue until all empty cells are filled and are valid.

This algorithm is intentionally inefficient as there are cases where many empty cells are filled but ultimately are invalid, causing many unnecessary iterations and branching trees. In addition, the algoirthm blindly fills in empty cells without much context of the rest of the board until it goes to check its

validity on every single iteration. This is good for our case, as maximizing the number of average branches needed provides a good idea of how the soft core porcessor may operate under prolonged load. With more difficult problem sets, the number of potential branches increases at a non-linear rate, almost exponentially seeming rate.

This algorithm is deterministic, meaning that no matter which hardware this algoirthm is used on, there should always the same number of iterations the program needs to solve the same problem set and the same final result. This aids in making fair comparisons between various implementations of the MicroBlaze processor.

III. IMPLEMENTATION ON NEXYS A7

The implementation of the Sudoku algorithm on the Nexys A7 FPGA board consisted of the MicroBlaze-V softcore processor, an AXI timer to create a timer for tracking how long the algorithm takes to complete, a UARTlite module to display messages and our benchmark results in a serial monitor, and an AXI GPIO module to allow the user to use the buttons on the board. For our benchmarking, we used four different configurations for MicroBlaze-V, which are the performance, throughput, area, and frequency configurations. The performance configuration uses a five-stage pipeline. The throughput configuration uses a four-stage pipeline, which can result in lower system frequency than the five-stage pipeline. The area configuration uses a three-stage pipeline with lower instruction throughput. The frequency configuration uses an eight-stage pipeline, which is meant to optimize frequency.

Using the AXI Timer, we tracked the number of clock cycles it takes for the backtracking algorithm to run and the total time was determined by dividing the number of clock cycles by 100,000,000 since the Nexys A7 boards runs on a 100 MHz clock. UARTlite was used to display messages in a PuTTY terminal, which ran at a baud rate of 9600. Our throughput benchmark was determined by the number of operations divided by the total time for the algorithm to run. In our case, operations represent each time a new digit is tested in an empty cell. For example, if the algorithm tests digits 1-9 for an empty cell, that would count as 9 operations.

IV. RESULTS

Our benchmarking was performed by looking at 3 key components. First, the number of clock cycles to perform the algorithm. Second, the total time it took to complete the algorithm. Lastly, we looked at the throughput of each configuration. We used the performance configuration as our base/reference so all of our results are normalized relative to the base case.

Looking at Fig. 2., we can see that the number of clock cycles is relatively similar for the performance, throughput, and area configurations. However, we can see a very noticeable difference for the frequency configuration. In this specific benchmark, the higher the value, the worse the performance of that specific configuration as it represents the algorithm taking more cycles to complete. We can see that the frequency configuration has the worst performance as it has the highest value on the hard and medium difficulties while throughput has the best performance as it has the lowest value in the same difficulties.

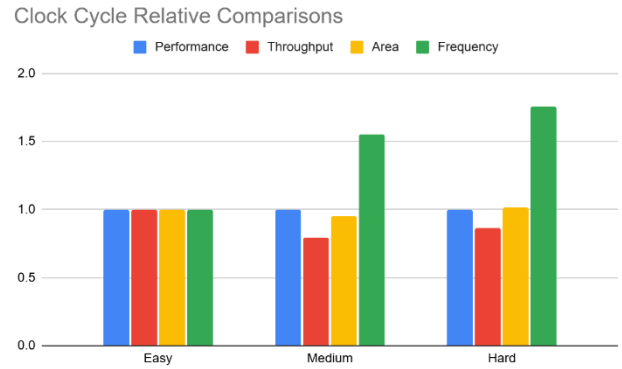

Fig. 2. Comparison of clock cycles for different problem sets and implementations

There is a correlation between the number of clock cycles and the total time for the algorithm to run (Fig. 3). Since the Nexys A7-100T is running on a 100 MHz clock, one second will equal 100,000,000 clock cycles, meaning we can divide the number of clock cycles by 100,000,000 to get the time for the program to run. This is why the results for the total time to run is identical to the number of clock cycles. Again, the lower the value, the better the results are since it takes less time to run.

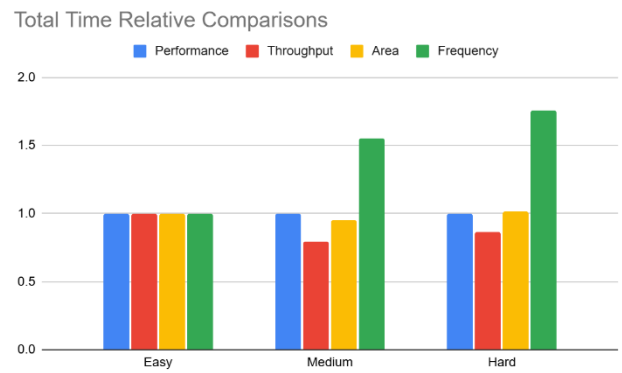

Fig. 3. Comparison of total program time for different problem sets and implementations

In Fig. 4., we are looking at the throughput of the algorithm. Throughput is defined as the number of operations divided by a unit of time. In our case, we are taking the number of operations and dividing it by the total time in seconds. This means that the higher values represent a greater throughput, meaning better performance. We can see that there is no difference on the easy difficulty, but in the medium and hard difficulties, we can see that the throughput configuration produced the highest throughput while the frequency configuration produced the lowest throughput.

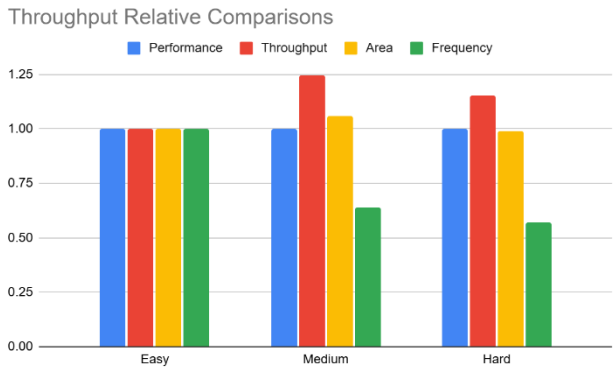

Fig. 4. Comparison of throughput for different problem sets and implementations

In Fig. 5., we are looking at the amount of resources, which consists of look-up tables and flip-flops, used for each type of implementation. We can see that frequency utilized the most amount of resources while producing the worst results, while area had the least amount of resources used. We do see that the performance, throughput, and area configurations had similar number of resources utilized while producing better results than the frequency configuration.

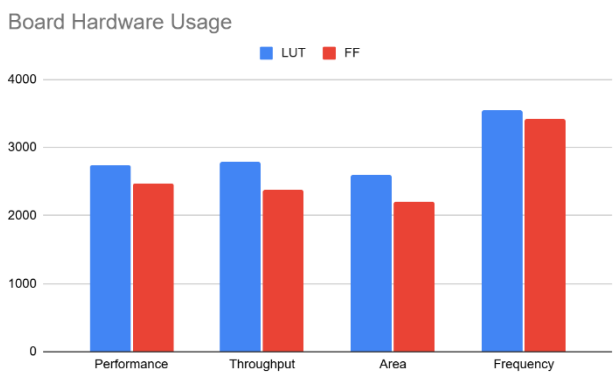

Fig. 5. Comparison of hardware requirements for different implementations

V. CONCLUSION

Overall, we were able to determine that using different implementations of MicroBlaze-V affected not only the amount of resources used on the FPGA, but also our benchmark results. We saw that the frequency configuration for MicroBlaze-V used the most amount of resources while producing the worst results out of the four configurations. The throughput configuration provided the best results out of the four configurations, while having similar resource usage to the the performance and area configurations.

REFERENCES

[1] Taylor, Adam. "MicroZed Chronicles: MicroBlaze v MCS." *Adiuvo Engineering Website*, 12 June 2024, www.adiuvoengineering.com/post/microzed-chronicles-microblaze-v-mcs.